Python Strings Guide

You said:

complete string all concept in python from basics to advanced

ChatGPT said:

Python Strings: Complete Guide from Basics to Advanced

A Python string is an immutable sequence of Unicode characters. Strings are used for names, messages, passwords, file paths, URLs, documents, NLP, data cleaning, APIs, and almost every Python project.

1. Creating Strings

Python represents strings using the `str` data type.

```
name = "Aamir"
city = 'Jaipur'
message = """This is
a multiline
string."""

print(name)
print(type(name))
```

Output:

```
Aamir
<class 'str'>
```

Empty string

```
text = ""
print(len(text))  # 0
```

An empty string contains no character. It is different from `None`.

```
empty = ""
missing = None

print(type(empty))    # str
print(type(missing))  # NoneType
```

Convert another value into a string

```
age = 24
salary = 45000.50

age_text = str(age)
salary_text = str(salary)

print(age_text, type(age_text))
print(salary_text, type(salary_text))
```

2. Quotes Inside Strings

Use different outer quotes:

```
sentence1 = "I'm learning Python."
sentence2 = 'He said, "Python is easy."'

print(sentence1)
print(sentence2)
```

Or escape the quote using \:

```
sentence = 'I\'m learning Python.'
quote = "He said, \"Hello!\""

print(sentence)
print(quote)
```

3. Escape Sequences

Escape sequences represent special characters.

Escape	Meaning
\n	New line
\t	Tab space
\\	Backslash
\'	Single quote
\"	Double quote
\r	Carriage return
\b	Backspace
\uXXXX	Unicode character

```
print("Python\nMachine Learning")
print("Name:\tAamir")
print("C:\\Users\\Aamir")
print("\u2764")  # Heart
```

Output:

```
Python
Machine Learning
Name:   Aamir
C:\Users\Aamir
❤
```

Raw strings

A raw string treats backslashes mostly as normal characters.

```
normal_path = "C:\\Users\\Aamir\\Documents"
raw_path = r"C:\Users\Aamir\Documents"

print(normal_path)
print(raw_path)
```

Raw strings are useful for:

- Windows paths
- Regular expressions
- Text containing many backslashes

A raw string cannot end with a single backslash:

```
# Invalid
# path = r"C:\Users\"
```

---

4. Strings Are Sequences

Every character has an index.

```
String:  P  Y  T  H  O  N
Index:   0  1  2  3  4  5
Negative:-6 -5 -4 -3 -2 -1
```

Positive indexing

```
language = "Python"

print(language[0])  # P
print(language[3])  # h
```

Negative indexing

```
print(language[-1])  # n
print(language[-2])  # o
```

Index out of range

```
language = "Python"

# print(language[10])
# IndexError: string index out of range
```

Safe checking:

```
index = 10

if 0 <= index < len(language):
    print(language[index])
else:
    print("Invalid index")
```

5. String Slicing

Slicing extracts a part of a string.

```
string[start:stop:step]
```

- start: starting index, included
- stop: ending index, excluded
- step: number of positions to move

```
text = "Python Programming"

print(text[0:6])   # Python
print(text[7:18])  # Programming
print(text[:6])    # Python
print(text[7:])    # Programming
print(text[:])     # Complete string
```

Why is the stop index excluded?

```
text = "Python"
result = text[1:4]
```

Dry run:

Index	Character	Selected?
0	P	No
1	y	Yes
2	t	Yes
3	h	Yes

Index	Character	Selected?
4	o	No—stop here

Result:

```
yth
```

Slicing with steps

```
text = "ABCDEFGHIJ"

print(text[0:10:2])  # ACEGI
print(text[1::2])    # BDFHJ
```

Reverse a string

```
text = "Python"
reversed_text = text[::-1]

print(reversed_text)  # nohtyP
```

Negative slicing

```
text = "Programming"

print(text[-6:])   # amming
print(text[:-3])   # Programm
print(text[-1:-6:-1])  # gnimm
```

6. String Immutability

Strings are immutable. After creation, their characters cannot be modified directly.

```
name = "Aamir"

# Invalid:
# name[0] = "S"
# TypeError: 'str' object does not support item assignment
```

Create a new string instead:

```
name = "Aamir"
new_name = "S" + name[1:]

print(new_name)  # Samir
```

Another solution:

```
name = "Aamir"
characters = list(name)
characters[0] = "S"
new_name = "".join(characters)

print(new_name)
```

Important idea

```
text = "hello"
print(id(text))

text = text.upper()
print(id(text))
```

`upper()` does not change the old string. It creates and returns a new string.

---

7. Length of a String

Use `len()`:

```
text = "Machine Learning"

print(len(text))  # 16
```

Spaces are also counted.

```
print(len("AI ML"))  # 5
```

---

8. String Operators

Concatenation: +

```
first_name = "Aamir"
last_name = "Azad"

full_name = first_name + " " + last_name
print(full_name)
```

Repetition: *

```
print("Python " * 3)
print("-" * 30)
```

Membership: in

```
text = "Python Programming"

print("Python" in text)       # True
print("Java" in text)         # False
print("programming" in text)  # False: case-sensitive
```

Non-membership: not in

```
print("Java" not in text)  # True
```

9. String Comparison

Strings are compared lexicographically, using Unicode values.

```
print("apple" == "apple")  # True
print("apple" != "Apple")  # True
print("apple" < "banana")   # True
print("A" < "a")           # True
```

Check Unicode values:

```
print(ord("A"))  # 65
print(ord("a"))  # 97
print(chr(65))   # A
```

Case-insensitive comparison

```
first = "PYTHON"
second = "python"

print(first.lower() == second.lower())  # True
```

For reliable international text comparison, use casefold():

```
first = "Straße"
second = "STRASSE"

print(first.lower() == second.lower())      # False
print(first.casefold() == second.casefold())  # True
```

10. Iterating Through a String

Using for

```
name = "Aamir"

for character in name:
    print(character)
```

Using index

```
for index in range(len(name)):
    print(index, name[index])
```

Using enumerate()

```
for index, character in enumerate(name):
    print(index, character)
```

Output:

```
0 A
1 a
2 m
3 i
4 r
```

Using while

```
text = "Python"
index = 0

while index < len(text):
    print(text[index])
    index += 1
```

11. Changing Letter Case

lower()

```
text = "PYTHON Programming"
print(text.lower())
```

upper()

```
print(text.upper())
```

capitalize()

Makes the first character uppercase and the remaining characters lowercase.

```
print("python PROGRAMMING".capitalize())
# Python programming
```

title()

Capitalizes the first letter of every word.

```
print("python machine learning".title())
# Python Machine Learning
```

Be careful with apostrophes:

```
print("aamir's project".title())
# Aamir'S Project
```

swapcase()

```
print("Python ML".swapcase())
# pYTHON ml
```

casefold()

Aggressive lowercasing for case-insensitive comparisons.

```
email1 = "AAMIR@EXAMPLE.COM"
email2 = "aamir@example.com"

print(email1.casefold() == email2.casefold())
```

12. Removing Spaces and Characters

strip()

Removes characters from both ends.

```
text = "   Python   "
print(text.strip())  # Python
```

lstrip()

```
print(text.lstrip())  # "Python   "
```

rstrip()

```
print(text.rstrip())  # "   Python"
```

Removing specified characters

```
text = "***Python***"

print(text.strip("*"))   # Python
print(text.lstrip("*"))  # Python***
print(text.rstrip("*"))  # ***Python
```

Important: strip() treats its argument as a set of characters, not a complete word.

```
text = "xyxPythonxy"

print(text.strip("xy"))
# Python
```

To remove an exact prefix or suffix, use the following methods.

removeprefix()

```
url = "https://example.com"
print(url.removeprefix("https://"))
```

removesuffix()

```
filename = "report.pdf"
print(filename.removesuffix(".pdf"))
```

13. Searching Inside Strings

find()

Returns the first index, or -1 if not found.

```
text = "Python Programming Python"

print(text.find("Python"))  # 0
print(text.find("Java"))    # -1
```

With a search range:

```
print(text.find("Python", 1))  # 19
```

rfind()

Searches from the right.

```
print(text.rfind("Python"))  # 19
```

index()

Similar to find(), but raises ValueError when not found.

```
print(text.index("Programming"))  # 7

# print(text.index("Java"))
# ValueError: substring not found
```

rindex()

Returns the last occurrence, but raises an error if not found.

```
print(text.rindex("Python"))  # 19
```

count()

```
text = "banana"

print(text.count("a"))   # 3
print(text.count("an"))  # 2
```

count() counts non-overlapping occurrences:

```
print("aaaa".count("aa"))  # 2
```

14. Checking Prefix and Suffix

startswith()

```
url = "https://example.com"

print(url.startswith("https"))  # True
print(url.startswith("http"))   # True
```

Check multiple prefixes:

```
filename = "photo.png"

print(filename.startswith(("img_", "photo", "pic_")))
```

endswith()

```
print(filename.endswith(".png"))  # True
print(filename.endswith((".jpg", ".jpeg", ".png")))  # True
```

15. Replacing Characters or Words

replace()

```
text = "I like Java"
new_text = text.replace("Java", "Python")

print(new_text)
```

Limit replacements:

```
text = "one one one"
print(text.replace("one", "two", 2))
# two two one
```

Strings remain unchanged:

```
original = "Java"
updated = original.replace("Java", "Python")

print(original)  # Java
print(updated)   # Python
```

16. Splitting Strings

split()

```
sentence = "Python is easy to learn"
words = sentence.split()

print(words)
```

Output:

```
['Python', 'is', 'easy', 'to', 'learn']
```

Split using a delimiter:

```
data = "Aamir,24,AI Engineer"
values = data.split(",")

print(values)
```

Use maxsplit:

```
text = "one-two-three-four"
print(text.split("-", 2))
# ['one', 'two', 'three-four']
```

rsplit()

Splits from the right when maxsplit is provided.

```
path = "home/user/documents/report.pdf"

print(path.rsplit("/", 1))
# ['home/user/documents', 'report.pdf']
```

splitlines()

```
text = "Python\nJava\nSQL"
print(text.splitlines())
```

17. Joining Strings

join() combines iterable elements into one string.

```
words = ["Python", "is", "powerful"]

sentence = " ".join(words)
print(sentence)
```

Other separators:

```
print("-".join(words))
print(", ".join(words))
print("\n".join(words))
```

All elements must be strings:

```
numbers = [10, 20, 30]

# Invalid:
# print(",".join(numbers))

result = ",".join(map(str, numbers))
print(result)
```

Dry run:

```
words = ["AI", "ML", "Python"]
result = " | ".join(words)
```

Step	Current result
1	AI
2	AI | ML
3	AI | ML | Python

18. partition() and rpartition()

partition()

Splits into exactly three parts:

```
email = "aamir@example.com"

username, separator, domain = email.partition("@")

print(username)   # aamir
print(separator)  # @
print(domain)     # example.com
```

If the separator is missing:

```
print("aamir".partition("@"))
# ('aamir', '', '')
```

rpartition()

Splits at the last occurrence.

```
filename = "project.backup.zip"
print(filename.rpartition("."))
# ('project.backup', '.', 'zip')
```

19. Character Validation Methods

isalpha()

True when all characters are alphabetic.

```
print("Aamir".isalpha())      # True
print("Aamir123".isalpha())   # False
print("Aamir Azad".isalpha()) # False
```

isdigit()

```
print("12345".isdigit()) # True
print("12.5".isdigit())  # False
print("-20".isdigit())   # False
```

isdecimal()

Checks decimal characters.

```
print("123".isdecimal()) # True
print("²".isdecimal())   # False
```

isnumeric()

Supports a wider range of numeric characters.

```
print("123".isnumeric()) # True
print("½".isnumeric())   # True
```

Relationship:

```
isdecimal() ⊆ isdigit() ⊆ isnumeric()
```

isalnum()

```
print("Aamir123".isalnum())  # True
print("Aamir 123".isalnum()) # False
```

isspace()

```
print("   ".isspace())   # True
print("\n\t".isspace())  # True
```

islower() and isupper()

```
print("python123".islower())  # True
print("PYTHON123".isupper())  # True
```

istitle()

```
print("Python Machine Learning".istitle())  # True
```

isidentifier()

Checks whether text can be a valid Python identifier.

```
print("student_name".isidentifier())  # True
print("2student".isidentifier())      # False
print("student-name".isidentifier())  # False
```

It does not check whether the word is a Python keyword:

```
import keyword

word = "class"

print(word.isidentifier())    # True
print(keyword.iskeyword(word))  # True
```

isascii()

```
print("Python123".isascii())  # True
print("नमस्ते".isascii())     # False
```

isprintable()

```
print("Hello!".isprintable())    # True
print("Hello\n".isprintable())   # False
```

20. Alignment and Padding

center()

```
title = "PYTHON"

print(title.center(20, "-"))
# -------PYTHON-------
```

ljust()

```
print("Aamir".ljust(10, "."))
# Aamir.....
```

rjust()

```
print("500".rjust(10, "0"))
# 0000000500
```

zfill()

Adds zeros while respecting the sign.

```
print("42".zfill(5))   # 00042
print("-42".zfill(5))  # -0042
```

21. String Formatting

Old % formatting

```
name = "Aamir"
age = 24

print("My name is %s and I am %d years old." % (name, age))
```

Common placeholders:

Placeholder	Meaning
%s	String
%d	Integer
%f	Float
%.2f	Float with two decimal places

```
price = 99.4567
print("Price: %.2f" % price)
```

str.format()

```
name = "Aamir"
role = "AI/ML Engineer"

print("My name is {} and I am an {}.".format(name, role))
```

Positioned values:

```
print("{1} is learning {0}".format("Python", "Aamir"))
```

Named values:

```
print(
    "Name: {name}, Score: {score}".format(
        name="Aamir",
        score=87.6
    )
)
```

F-strings

F-strings are generally the clearest modern approach.

```
name = "Aamir"
age = 24

print(f"My name is {name} and I am {age} years old.")
```

Expressions inside f-strings:

```
a = 10
b = 20

print(f"Total = {a + b}")
print(f"Maximum = {max(a, b)}")
```

Number formatting

```
price = 1250000.5678
percentage = 0.8567

print(f"{price:.2f}")     # 1250000.57
print(f"{price:,.2f}")    # 1,250,000.57
print(f"{percentage:.2%}")  # 85.67%
```

Width and alignment

```
name = "Aamir"

print(f"|{name:<10}|")  # Left
print(f"|{name:>10}|")  # Right
print(f"|{name:^10}|")  # Center
```

Sign and zero padding

```
number = 42

print(f"{number:+d}")   # +42
print(f"{number:05d}")  # 00042
```

Binary, octal and hexadecimal

```
number = 20

print(f"{number:b}")  # 10100
print(f"{number:o}")  # 24
print(f"{number:x}")  # 14
```

Debugging syntax

```
score = 92
print(f"{score=}")
# score=92
```

!r, !s and !a

```
text = "Python\nML"

print(f"{text!s}")  # str representation
print(f"{text!r}")  # repr representation
print(f"{text!a}")  # ASCII representation
```

22. str() Versus repr()

str() creates a user-friendly representation.
repr() creates a developer-oriented representation.

```
text = "Python\nML"

print(str(text))
print(repr(text))
```

Output:

```
Python
ML
'Python\nML'
```

23. Translation and Bulk Character Replacement

maketrans() and translate()

```
translation = str.maketrans({
    "a": "@",
    "i": "1",
    "o": "0"
})

text = "Aamir is learning Python"
print(text.translate(translation))
```

Delete characters:

```
delete_vowels = str.maketrans("", "", "aeiouAEIOU")

text = "Machine Learning"
print(text.translate(delete_vowels))
# Mchn Lrnng
```

For many single-character replacements, translate() is usually cleaner than repeatedly calling replace().

24. Unicode Strings

Python 3 strings use Unicode, which supports many writing systems and symbols.

```
english = "Hello"
hindi = "नमस्ते"
arabic = "مرحبا"
emoji = "🤖"

print(english, hindi, arabic, emoji)
```

Unicode normalization

Some visually identical characters can have different internal representations.

```
import unicodedata

first = "é"
second = "e\u0301"

print(first == second)  # False

first_normalized = unicodedata.normalize("NFC", first)
second_normalized = unicodedata.normalize("NFC", second)

print(first_normalized == second_normalized)  # True
```

Common normalization forms:

- NFC: composed standard form
- NFD: decomposed form
- NFKC: compatibility-composed form
- NFKD: compatibility-decomposed form

---

25. Strings and Bytes

A string represents human-readable text. Bytes represent raw binary data.

```
text = "Python"
binary_data = text.encode("utf-8")

print(text)              # Python
print(binary_data)       # b'Python'
print(type(binary_data))  # bytes
```

Decode bytes into a string

```
decoded_text = binary_data.decode("utf-8")

print(decoded_text)
print(type(decoded_text))
```

Process:

```
String --encode()--> Bytes --decode()--> String
```

Example with Hindi:

```
message = "नमस्ते"

encoded = message.encode("utf-8")
decoded = encoded.decode("utf-8")

print(encoded)
print(decoded)
```

Always use the same compatible encoding for encoding and decoding.

---

26. Copying and String Interning

Because strings are immutable, assigning one string to another is safe:

```
first = "Python"
second = first

print(first is second)
```

Python may reuse some immutable string objects through string interning. Therefore, never use `is` for value comparison.

```
first = "Python"
second = "Python"

# Correct
print(first == second)

# Not recommended for value comparison
print(first is second)
```

- `==` compares values
- `is` compares object identities

---

27. Useful Built-in Functions

```
text = "Python"

print(len(text))       # 6
print(min(text))       # P
print(max(text))       # y
print(sorted(text))    # ['P', 'h', 'n', 'o', 't', 'y']
print(list(text))      # ['P', 'y', 't', 'h', 'o', 'n']
print(tuple(text))     # ('P', 'y', 't', 'h', 'o', 'n')
```

Sorting characters into a string

```
text = "python"
sorted_text = "".join(sorted(text))

print(sorted_text)  # hnopty
```

28. All Major String Methods

Category	Methods
Case conversion	lower, upper, capitalize, title, swapcase, casefold
Search	find, rfind, index, rindex, count
Prefix/suffix	startswith, endswith, removeprefix, removesuffix
Validation	isalpha, isdigit, isdecimal, isnumeric, isalnum, isspace, islower, isupper, istitle, isidentifier, isascii, isprintable
Whitespace removal	strip, lstrip, rstrip
Splitting	split, rsplit, splitlines, partition, rpartition
Combining	join
Modification	replace, translate, maketrans
Alignment	center, ljust, rjust, zfill
Formatting	format, format_map

Category	Methods
Encoding	encode
Tabs	expandtabs

expandtabs()

```
text = "Name\tRole\tCity"

print(text.expandtabs(15))
```

format_map()

```
student = {
    "name": "Aamir",
    "score": 87.6
}

result = "Name: {name}, Score: {score}".format_map(student)
print(result)
```

29. Common String Programs

Program 1: Reverse a string

```
def reverse_string(text):
    return text[::-1]


print(reverse_string("Python"))
```

Alternative without slicing:

```
def reverse_string_manual(text):
    result = ""

    for character in text:
        result = character + result

    return result


print(reverse_string_manual("Python"))
```

Dry run for "ABC":

Character	result before	character + result
A	""	A
B	A	BA
C	BA	CBA

Program 2: Palindrome check

A palindrome reads the same forward and backward.

Examples: madam, level, racecar.

```
def is_palindrome(text):
    cleaned = "".join(
        character.casefold()
        for character in text
        if character.isalnum()
    )

    return cleaned == cleaned[::-1]


print(is_palindrome("Madam"))                    # True
print(is_palindrome("A man, a plan, a canal: Panama"))  # True
```

Two-pointer version:

```
def is_palindrome_two_pointer(text):
    cleaned = "".join(
        character.casefold()
        for character in text
        if character.isalnum()
    )

    left = 0
    right = len(cleaned) - 1

    while left < right:
        if cleaned[left] != cleaned[right]:
            return False

        left += 1
        right -= 1

    return True
```

Complexity:

- Time: (O(n))
- Space: (O(n)) because of cleaned string

Program 3: Count vowels and consonants

```
def count_vowels_consonants(text):
    vowels = set("aeiou")
    vowel_count = 0
    consonant_count = 0

    for character in text.casefold():
        if character.isalpha():
            if character in vowels:
                vowel_count += 1
            else:
                consonant_count += 1

    return vowel_count, consonant_count


vowels, consonants = count_vowels_consonants("Machine Learning")

print("Vowels:", vowels)
print("Consonants:", consonants)
```

Program 4: Character frequency

```
def character_frequency(text):
    frequency = {}

    for character in text:
        frequency[character] = frequency.get(character, 0) + 1

    return frequency


print(character_frequency("banana"))
```

Output:

```
{'b': 1, 'a': 3, 'n': 2}
```

Using Counter:

```
from collections import Counter

frequency = Counter("banana")
print(frequency)
```

Program 5: First non-repeating character

```
from collections import Counter


def first_non_repeating(text):
    frequency = Counter(text)

    for character in text:
        if frequency[character] == 1:
            return character

    return None


print(first_non_repeating("aabbcddee"))  # c
```

Complexity:

- Time: (O(n))
- Space: (O(k)), where k is the number of unique characters

Program 6: Check anagrams

Anagrams contain the same characters with the same frequencies.

Examples: listen and silent.

```
from collections import Counter


def are_anagrams(first, second):
    clean_first = "".join(
        character.casefold()
        for character in first
        if character.isalnum()
    )

    clean_second = "".join(
        character.casefold()
        for character in second
        if character.isalnum()
    )

    return Counter(clean_first) == Counter(clean_second)


print(are_anagrams("Listen", "Silent"))       # True
print(are_anagrams("Dormitory", "Dirty room"))  # True
```

Sorting approach:

```
def are_anagrams_sorting(first, second):
    return sorted(first.casefold()) == sorted(second.casefold())
```

Complexity:

- Counter solution: (O(n))
- Sorting solution: (O(n \log n))

Program 7: Count words

```
def count_words(sentence):
    return len(sentence.split())


print(count_words("Python is easy to learn"))  # 5
```

For complicated punctuation and Unicode text, regular expressions or NLP libraries may be more suitable.

Program 8: Find duplicate characters

```
from collections import Counter


def duplicate_characters(text):
    frequency = Counter(text.casefold())

    return {
        character: count
        for character, count in frequency.items()
        if count > 1 and not character.isspace()
    }


print(duplicate_characters("Programming"))
```

Program 9: Remove duplicate characters

Preserve the original order:

```
def remove_duplicates(text):
    seen = set()
    result =

    for character in text:
        if character not in seen:
            seen.add(character)
            result.append(character)

    return "".join(result)


print(remove_duplicates("programming"))
# progamin
```

Short approach:

```
print("".join(dict.fromkeys("programming")))
```

Program 10: Capitalize every word safely

```
def capitalize_words(sentence):
    return " ".join(
        word[:1].upper() + word[1:].lower()
        for word in sentence.split()
    )


print(capitalize_words("python MACHINE learning"))
```

Program 11: String compression

Input:

```
aaabbccccd
```

Output:

```
a3b2c4d1
```

```
def compress_string(text):
    if not text:
        return ""

    result =
    count = 1

    for index in range(1, len(text)):
        if text[index] == text[index - 1]:
            count += 1
        else:
            result.append(text[index - 1] + str(count))
            count = 1

    result.append(text[-1] + str(count))
    return "".join(result)


print(compress_string("aaabbccccd"))
```

Dry run:

Current	Previous	Count	Result
a	a	2	``
a	a	3	``
b	a	1	['a3']
b	b	2	['a3']
c	b	1	['a3', 'b2']
c	c	2	unchanged
c	c	3	unchanged
c	c	4	unchanged
d	c	1	add c4
End	d	1	add d1

30. Substring Searching

Python approach

```
text = "machine learning"
pattern = "learn"

print(pattern in text)
print(text.find(pattern))
```

Naive substring search

```
def naive_search(text, pattern):
    if pattern == "":
        return 0

    for start in range(len(text) - len(pattern) + 1):
        matched = True

        for offset in range(len(pattern)):
            if text[start + offset] != pattern[offset]:
                matched = False
                break

        if matched:
            return start

    return -1


print(naive_search("machine learning", "learn"))
```

Complexity:

- Worst-case time: (O(nm))
- n: length of text
- m: length of pattern

---

31. Longest Common Prefix

```
def longest_common_prefix(words):
    if not words:
        return ""

    prefix = words[0]

    for word in words[1:]:
        while not word.startswith(prefix):
            prefix = prefix[:-1]

            if not prefix:
                return ""

    return prefix


print(longest_common_prefix(["flower", "flow", "flight"]))
# fl
```

32. Longest Substring Without Repeating Characters

This is an important sliding-window interview problem.

```
def longest_unique_substring(text):
    last_seen = {}
    left = 0
    best_start = 0
    best_length = 0

    for right, character in enumerate(text):
        if character in last_seen and last_seen[character] >= left:
            left = last_seen[character] + 1

        last_seen[character] = right

        current_length = right - left + 1

        if current_length > best_length:
            best_length = current_length
            best_start = left

    substring = text[best_start:best_start + best_length]
    return substring, best_length


print(longest_unique_substring("abcabcbb"))
# ('abc', 3)
```

Dry run:

Right	Character	Left	Current window	Best
0	a	0	a	a
1	b	0	ab	ab
2	c	0	abc	abc
3	a	1	bca	abc
4	b	2	cab	abc
5	c	3	abc	abc
6	b	5	cb	abc
7	b	7	b	abc

Complexity:

- Time: (O(n))

- Space: (O(k))

33. Regular Expressions

Regular expressions are patterns used to search, validate, split, and replace text.

```
import re
```

Common regex symbols

Pattern	Meaning
.	Any character except newline
\d	Digit
\D	Non-digit
\w	Word character
\W	Non-word character
\s	Whitespace
\S	Non-whitespace
^	Start of string
$	End of string
*	Zero or more
+	One or more
?	Zero or one
{n}	Exactly n times
{m,n}	Between m and n times
[abc]	a, b, or c
[^abc]	Anything except a, b, c
()	Capturing group

Pattern	Meaning
\b	Word boundary
|	OR

Use raw strings for regex patterns:

```
pattern = r"\d+"
```

re.search()

Finds the first match anywhere.

```
text = "Order number is 45872"
match = re.search(r"\d+", text)

if match:
    print(match.group())  # 45872
```

re.match()

Checks from the beginning.

```
print(re.match(r"Python", "Python is easy"))
print(re.match(r"Python", "Learn Python"))
```

re.fullmatch()

Checks the complete string.

```
phone = "9876543210"

if re.fullmatch(r"[6-9]\d{9}", phone):
    print("Valid Indian mobile number")
else:
    print("Invalid mobile number")
```

re.findall()

```
text = "Products cost 100, 250 and 500 rupees"
numbers = re.findall(r"\d+", text)

print(numbers)
```

re.finditer()

```
for match in re.finditer(r"\d+", text):
    print(match.group(), match.start(), match.end())
```

re.sub()

```
text = "My phone number is 9876543210"
hidden = re.sub(r"\d", "*", text)

print(hidden)
```

re.split()

```
text = "Python,Java;SQL Machine-Learning"
words = re.split(r"[,;\s-]+", text)

print(words)
```

Capturing groups

```
date = "12-08-2026"
match = re.fullmatch(r"(\d{2})-(\d{2})-(\d{4})", date)

if match:
    day, month, year = match.groups()
    print(day, month, year)
```

Named groups

```
pattern = r"(?P<day>\d{2})-(?P<month>\d{2})-(?P<year>\d{4})"
match = re.fullmatch(pattern, "12-08-2026")

if match:
    print(match.groupdict())
```

Basic email validation

```
def is_valid_email(email):
    pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
    return re.fullmatch(pattern, email) is not None


print(is_valid_email("aamir@example.com"))
```

This is useful for basic input checking, but complete email-address validation is more complicated.

34. Cleaning Real-World Text

```
import re
import unicodedata


def clean_text(text):
    # Normalize Unicode
    text = unicodedata.normalize("NFKC", text)

    # Remove spaces from both ends
    text = text.strip()

    # Convert repeated whitespace into one space
    text = re.sub(r"\s+", " ", text)

    # Apply strong lowercase conversion
    text = text.casefold()

    return text


dirty_text = "   PYTHON\t\tMachine\nLearning   "
print(clean_text(dirty_text))
```

Output:

```
python machine learning
```

35. Performance and Memory

Avoid repeated concatenation in large loops

Less efficient:

```
result = ""

for number in range(10000):
    result += str(number)
```

Preferred:

```
parts =

for number in range(10000):
    parts.append(str(number))

result = "".join(parts)
```

Why?

Strings are immutable. Repeated `+=` operations may repeatedly create new string objects and copy existing content. A list plus `join()` is more predictable for large text construction.

Use `io.StringIO`

```
from io import StringIO

buffer = StringIO()

for number in range(5):
    buffer.write(f"Number: {number}\n")

result = buffer.getvalue()
buffer.close()

print(result)
```

StringIO is helpful when building large text dynamically.

---

36. String Operation Complexity

Let n be the string length.

Operation	Typical complexity
len(text)	(O(1))
text[index]	(O(1))
text[start:stop]	(O(k))
text1 + text2	(O(n+m))
character in text	(O(n))
find()	Usually (O(n)), implementation-dependent
replace()	(O(n))
split()	(O(n))
join()	(O(n)) for total output
lower() / upper()	(O(n))
Reverse with [::-1]	(O(n))
Character frequency	(O(n))

Here, k is the size of the produced slice, and m is the second string length.

---

37. Common Mistakes

Mistake 1: Modifying a string directly

```
text = "Python"
# text[0] = "J"
```

Correct:

```
text = "J" + text[1:]
```

Mistake 2: Forgetting that strings are case-sensitive

```
print("python" == "Python")  # False
```

Correct:

```
print("python".casefold() == "Python".casefold())
```

Mistake 3: Using `is` instead of `==`

```
# Wrong for value comparison
first is second
```

Correct:

```
first == second
```

Mistake 4: Assuming `find()` raises an error

```
index = "Python".find("Java")
print(index)  # -1
```

`index()` raises `ValueError`; `find()` returns `-1`.

Mistake 5: Forgetting to save a method's result

```
name = "aamir"
name.upper()

print(name)  # aamir
```

Correct:

```
name = name.upper()
```

Mistake 6: Passing integers to join()

```
numbers = [1, 2, 3]
result = ",".join(map(str, numbers))
```

Mistake 7: Confusing strip() with exact prefix removal

```
text = "www.example.com"

# Character-set removal
print(text.strip("www."))

# Exact prefix removal
print(text.removeprefix("www."))
```

Use removeprefix() when you mean one exact prefix.

Mistake 8: Using split(" ") for irregular whitespace

```
text = "Python    Machine\tLearning"

print(text.split(" "))  # May contain empty strings
print(text.split())     # Better
```

Mistake 9: Incorrect encoding and decoding

```
text = "नमस्ते"
data = text.encode("utf-8")
result = data.decode("utf-8")
```

Use compatible encodings.

Mistake 10: Writing regex without raw strings

Preferred:

```
pattern = r"\d+\s+\w+"
```

38. Interview Questions and Answers

1. What is a Python string?

A string is an immutable sequence of Unicode characters represented by the `str` class.

2. Are strings mutable?

No. Existing string characters cannot be modified. Every apparent modification creates a new string.

3. What is the difference between `find()` and `index()`?

- `find()` returns `-1` when a substring is absent.
- `index()` raises `ValueError`.

4. What is the difference between `split()` and `join()`?

- `split()` converts one string into a list of strings.
- `join()` combines multiple strings into one string.

5. What does `[::-1]` do?

It creates a reversed copy of a sequence.

6. What is the difference between `lower()` and `casefold()`?

`casefold()` performs stronger Unicode-aware case normalization and is better for case-insensitive comparison.

7. What is the difference between `==` and `is`?

- `==` compares values.
- `is` compares whether two variables refer to the same object.

8. What is the difference between strings and bytes?

- String: human-readable Unicode text
- Bytes: raw binary data

Use `encode()` to convert a string to bytes and `decode()` to convert bytes to a string.

9. How do you remove whitespace?

```
text.strip()
text.lstrip()
text.rstrip()
```

10. How do you count character frequencies?

```
from collections import Counter

frequency = Counter(text)
```

11. Why is join() preferred for large string construction?

Because repeated concatenation can create many temporary string objects. join() calculates and constructs the final result more efficiently.

12. How do you check a palindrome?

```
text == text[::-1]
```

For real-world input, first normalize case and remove spaces or punctuation.

13. How do you check an anagram?

Compare character frequencies:

```
Counter(first) == Counter(second)
```

14. How do you remove duplicate characters while preserving order?

```
result = "".join(dict.fromkeys(text))
```

15. Is Python indexing based on bytes or Unicode characters?

Python str indexing works with Unicode code points, not encoded UTF-8 bytes.

39. Complete Practice Program

```
import re
import unicodedata
from collections import Counter


def normalize_text(text):
    """Normalize Unicode, spaces and letter case."""
    text = unicodedata.normalize("NFKC", text)
    text = re.sub(r"\s+", " ", text.strip())
    return text.casefold()


def reverse_string(text):
    """Return the reversed string."""
    return text[::-1]


def is_palindrome(text):
    """Check a normalized alphanumeric palindrome."""
    cleaned = "".join(
        character
        for character in normalize_text(text)
        if character.isalnum()
    )
    return cleaned == cleaned[::-1]


def character_frequency(text):
    """Return character frequencies."""
    return Counter(text)


def word_frequency(text):
    """Count case-insensitive word frequencies."""
    words = re.findall(r"\b[\w']+\b", normalize_text(text))
    return Counter(words)


def are_anagrams(first, second):
    """Check whether two phrases are anagrams."""
    clean_first = "".join(
        character
        for character in normalize_text(first)
        if character.isalnum()
    )

    clean_second = "".join(
        character
```

```
        for character in normalize_text(second)
        if character.isalnum()
    )

    return Counter(clean_first) == Counter(clean_second)


def first_non_repeating(text):
    """Return the first non-repeating character."""
    frequency = Counter(text)

    for character in text:
        if frequency[character] == 1:
            return character

    return None


def remove_duplicate_characters(text):
    """Remove duplicate characters while preserving order."""
    return "".join(dict.fromkeys(text))


def compress_string(text):
    """Run-length encode consecutive characters."""
    if not text:
        return ""

    result =
    count = 1

    for index in range(1, len(text)):
        if text[index] == text[index - 1]:
            count += 1
        else:
            result.append(f"{text[index - 1]}{count}")
            count = 1

    result.append(f"{text[-1]}{count}")
    return "".join(result)


def longest_unique_substring(text):
    """Find the longest substring without repeated characters."""
    last_seen = {}
    left = 0
    best_start = 0
    best_length = 0

    for right, character in enumerate(text):
        if character in last_seen and last_seen[character] >= left:
```

```
            left = last_seen[character] + 1

        last_seen[character] = right
        current_length = right - left + 1

        if current_length > best_length:
            best_start = left
            best_length = current_length

    result = text[best_start:best_start + best_length]
    return result, best_length


def analyze_text(text):
    """Produce a basic text analysis report."""
    words = re.findall(r"\b[\w']+\b", text)

    vowels = sum(
        character.casefold() in "aeiou"
        for character in text
    )

    consonants = sum(
        character.isalpha()
        and character.casefold() not in "aeiou"
        for character in text
    )

    digits = sum(character.isdigit() for character in text)
    spaces = sum(character.isspace() for character in text)

    return {
        "original": text,
        "normalized": normalize_text(text),
        "characters": len(text),
        "words": len(words),
        "vowels": vowels,
        "consonants": consonants,
        "digits": digits,
        "spaces": spaces,
        "reversed": reverse_string(text),
        "word_frequency": word_frequency(text)
    }


def main():
    sample = "Python is powerful. Python is easy to learn!"

    report = analyze_text(sample)

    print("TEXT ANALYSIS")
```

```
    print("-" * 50)

    for key, value in report.items():
        print(f"{key:<15}: {value}")

    print("\nOTHER OPERATIONS")
    print("-" * 50)

    print("Palindrome:",
          is_palindrome("A man, a plan, a canal: Panama"))

    print("Anagrams:",
          are_anagrams("Dormitory", "Dirty room"))

    print("First unique:",
          first_non_repeating("aabbcddee"))

    print("Duplicates removed:",
          remove_duplicate_characters("programming"))

    print("Compressed:",
          compress_string("aaabbccccd"))

    substring, length = longest_unique_substring("abcabcbb")

    print("Longest unique substring:", substring)
    print("Longest unique length:", length)


if __name__ == "__main__":
    main()
```

40. Practice Problems

Beginner

1. Print every character with its index.
2. Reverse a string.
3. Count vowels and consonants.
4. Count uppercase and lowercase letters.
5. Remove spaces.
6. Replace one word with another.
7. Count the occurrence of a character.
8. Check whether a string starts or ends with specific text.
9. Convert a sentence into title case.

10. Extract the username and domain from an email.

Intermediate

1. Check a palindrome while ignoring punctuation.
2. Check two strings for anagrams.
3. Find duplicate characters.
4. Find the first non-repeating character.
5. Remove duplicate characters while preserving order.
6. Count word frequencies.
7. Find the longest word.
8. Compress consecutive characters.
9. Find all substring occurrences.
10. Find the longest common prefix.

Advanced

1. Longest substring without repeating characters.
2. Minimum window substring.
3. Group a list of anagrams.
4. Implement naive pattern matching.
5. Implement the KMP algorithm.
6. Find the longest palindromic substring.
7. Compare version-number strings.
8. Decode encoded text such as `3[a2[c]]`.
9. Implement string rotation checking.
10. Normalize and analyze multilingual Unicode text.

Final revision

Remember these core points:

- A string is an immutable Unicode sequence.
- Indexing accesses one character.
- Slicing extracts a sequence of characters.
- String methods return new values.
- `split()` breaks text; `join()` combines text.
- `find()` returns `-1`; `index()` raises an error.
- Use `casefold()` for strong case-insensitive comparisons.
- Use `==` for values and `is` for identities.
- Use `encode()` and `decode()` when converting between text and bytes.
- Use `join()` or `StringIO` when constructing large strings.

- Sliding windows, hash maps, two pointers, and frequency counters are the most important string interview patterns.

You said:

string with mean for revision

ChatGPT said:

Python String Quick Revision with Meanings

1. String meaning

A string is a sequence of characters enclosed in single, double, or triple quotes.

```
name = "Aamir"
language = 'Python'
message = """Python is
easy to learn."""
```

```
print(type(name))  # <class 'str'>
```

Strings may contain:

- Letters: "Python"
- Numbers as text: "123"
- Spaces: "Hello World"
- Symbols: "@#$"
- Unicode: "नमस्ते 🤖"

2. Important characteristics

Concept	Meaning
Ordered	Every character has a fixed position
Indexed	Characters can be accessed using index numbers
Immutable	Individual characters cannot be changed
Iterable	We can loop through characters

Concept	Meaning
Unicode	Supports English, Hindi, emojis and other languages

3. Indexing

Indexing means accessing one character using its position.

```
String:   P  y  t  h  o  n
Positive: 0  1  2  3  4  5
Negative: -6 -5 -4 -3 -2 -1
```

```
text = "Python"

print(text[0])   # P
print(text[2])   # t
print(text[-1])  # n
print(text[-2])  # o
```

4. Slicing

Slicing means extracting a part of a string.

```
string[start:stop:step]
```

The stop index is excluded.

```
text = "Python Programming"

print(text[0:6])   # Python
print(text[:6])    # Python
print(text[7:])    # Programming
print(text[::2])   # Pto rgamn
print(text[::-1])  # Reverse string
```

5. String immutability

Immutable means the existing string cannot be changed character by character.

```
name = "Aamir"

# name[0] = "S"  # TypeError
```

Create a new string:

```
name = "S" + name[1:]
print(name)  # Samir
```

6. String operators

Operator	Meaning	Example
+	Combine strings	"AI" + "ML"
*	Repeat a string	"Hi" * 3
in	Check presence	"Py" in "Python"
not in	Check absence	"Java" not in "Python"
==	Compare values	"AI" == "AI"
!=	Check different values	"AI" != "ML"

```
print("Aamir" + " Azad")      # Aamir Azad
print("Python " * 3)          # Python Python Python
print("Py" in "Python")       # True
print("Java" not in "Python")  # True
```

7. Important built-in functions

Function	Meaning
len(text)	Number of characters
str(value)	Converts value to string
min(text)	Smallest Unicode character

Function	Meaning
max(text)	Largest Unicode character
sorted(text)	Returns sorted character list
ord(char)	Returns Unicode number
chr(number)	Converts Unicode number to character

```
text = "Python"

print(len(text))      # 6
print(str(123))       # "123"
print(sorted(text))   # ['P', 'h', 'n', 'o', 't', 'y']
print(ord("A"))       # 65
print(chr(65))        # A
```

8. Case-conversion methods

Method	Meaning	Example result
lower()	Converts letters to lowercase	"PY".lower() → "py"
upper()	Converts letters to uppercase	"py".upper() → "PY"
capitalize()	First character uppercase	"python ML" → "Python ml"
title()	First letter of every word uppercase	"python ml" → "Python Ml"
swapcase()	Reverses uppercase/lowercase	"PyThOn" → "pYtHoN"
casefold()	Strong lowercase for comparison	Useful for Unicode

```
text = "python MACHINE learning"

print(text.lower())
print(text.upper())
print(text.capitalize())
print(text.title())
print(text.swapcase())
```

9. Space-removal methods

Method	Meaning
strip()	Removes spaces from both ends
lstrip()	Removes spaces from the left
rstrip()	Removes spaces from the right

```
text = "   Python   "

print(text.strip())   # Python
print(text.lstrip())  # Python___
print(text.rstrip())  # ___Python
```

Remove specified characters:

```
text = "***Python***"

print(text.strip("*"))   # Python
print(text.lstrip("*"))  # Python***
print(text.rstrip("*"))  # ***Python
```

10. Searching methods

Method	Meaning
find(value)	First index or -1
rfind(value)	Last index or -1
index(value)	First index or ValueError
rindex(value)	Last index or ValueError

Method	Meaning
count(value)	Number of occurrences

```
text = "Python Programming Python"

print(text.find("Python"))   # 0
print(text.rfind("Python"))  # 19
print(text.find("Java"))     # -1
print(text.count("Python"))  # 2
```

Important difference:

```
"Python".find("Java")   # -1
# "Python".index("Java")  # ValueError
```

11. Prefix and suffix methods

Method	Meaning
startswith()	Checks beginning
endswith()	Checks ending
removeprefix()	Removes exact beginning
removesuffix()	Removes exact ending

```
url = "https://example.com"

print(url.startswith("https"))              # True
print(url.endswith(".com"))                 # True
print(url.removeprefix("https://"))         # example.com
print("report.pdf".removesuffix(".pdf"))    # report
```

12. Replace and translate

replace()

Replaces old text with new text.

```
text = "I learn Java"
result = text.replace("Java", "Python")

print(result)  # I learn Python
```

Limit replacements:

```
print("one one one".replace("one", "two", 2))
# two two one
```

translate()

Replaces multiple characters efficiently.

```
table = str.maketrans({"a": "@", "i": "1"})

print("aamir".translate(table))
# @@m1r
```

13. Splitting and joining

split()

Converts a string into a list.

```
text = "Python is easy"

print(text.split())
# ['Python', 'is', 'easy']
```

```
data = "Aamir,24,AIML"

print(data.split(","))
# ['Aamir', '24', 'AIML']
```

rsplit()

Splits from the right when a limit is given.

```
path = "home/user/report.pdf"

print(path.rsplit("/", 1))
# ['home/user', 'report.pdf']
```

join()

Combines string elements.

```
words = ["Python", "is", "easy"]

print(" ".join(words))  # Python is easy
print("-".join(words))  # Python-is-easy
```

Joining numbers:

```
numbers = [10, 20, 30]

print(",".join(map(str, numbers)))
# 10,20,30
```

14. Partition methods

Method	Meaning
partition()	Splits at first separator into three parts
rpartition()	Splits at last separator into three parts

```
email = "aamir@example.com"

print(email.partition("@"))
# ('aamir', '@', 'example.com')
```

```
filename = "project.backup.zip"

print(filename.rpartition("."))
# ('project.backup', '.', 'zip')
```

15. Checking methods

These methods usually return True or False.

Method	Meaning
isalpha()	Contains only letters
isdigit()	Contains only digit characters
isdecimal()	Contains only decimal digits
isnumeric()	Contains numeric characters
isalnum()	Contains letters or numbers
isspace()	Contains only whitespace
islower()	All letters are lowercase
isupper()	All letters are uppercase
istitle()	Every word uses title case
isidentifier()	Valid Python identifier
isascii()	Contains only ASCII characters
isprintable()	Contains only printable characters

```
print("Aamir".isalpha())         # True
print("123".isdigit())           # True
print("Aamir123".isalnum())      # True
print("   ".isspace())           # True
print("python".islower())        # True
print("PYTHON".isupper())        # True
print("Python Course".istitle())  # True
print("student_name".isidentifier())  # True
```

Important:

```
print("-123".isdigit())  # False
print("12.5".isdigit())  # False
print("Aamir Azad".isalpha())  # False because of space
```

16. Alignment methods

Method	Meaning
center()	Places text in the center
ljust()	Places text on the left
rjust()	Places text on the right
zfill()	Adds zeros on the left

```
print("Python".center(14, "-"))  # ----Python----
print("Aamir".ljust(10, "."))   # Aamir.....
print("500".rjust(8, "0"))      # 00000500
print("42".zfill(5))            # 00042
```

17. String formatting

F-string

```
name = "Aamir"
score = 87.6

print(f"Name: {name}, Score: {score}")
```

Decimal places

```
price = 1546.789

print(f"{price:.2f}")   # 1546.79
print(f"{price:,.2f}")  # 1,546.79
```

Percentage

```
accuracy = 0.9567

print(f"{accuracy:.2%}")  # 95.67%
```

Alignment

```
print(f"|{'Python':<10}|")  # Left
print(f"|{'Python':>10}|")  # Right
print(f"|{'Python':^10}|")  # Center
```

18. Looping through strings

```
text = "AIML"

for character in text:
    print(character)
```

With index:

```
for index, character in enumerate(text):
    print(index, character)
```

Output:

```
0 A
1 I
2 M
3 L
```

19. Important interview programs

Reverse a string

```
text = "Python"
print(text[::-1])  # nohtyP
```

Palindrome

```
def is_palindrome(text):
    cleaned = "".join(
        char.casefold()
        for char in text
        if char.isalnum()
    )

    return cleaned == cleaned[::-1]


print(is_palindrome("Madam"))  # True
```

Anagram

```
from collections import Counter


def is_anagram(first, second):
    first = first.replace(" ", "").casefold()
    second = second.replace(" ", "").casefold()

    return Counter(first) == Counter(second)


print(is_anagram("listen", "silent"))  # True
```

Character frequency

```
def character_frequency(text):
    frequency = {}

    for character in text:
        frequency[character] = frequency.get(character, 0) + 1

    return frequency


print(character_frequency("banana"))
# {'b': 1, 'a': 3, 'n': 2}
```

Count vowels and consonants

```
def count_letters(text):
    vowels = 0
    consonants = 0

    for character in text.casefold():
        if character.isalpha():
            if character in "aeiou":
                vowels += 1
            else:
                consonants += 1

    return vowels, consonants


print(count_letters("Machine Learning"))
```

First non-repeating character

```
from collections import Counter


def first_unique(text):
    frequency = Counter(text)

    for character in text:
        if frequency[character] == 1:
            return character

    return None


print(first_unique("aabbcddee"))  # c
```

Remove duplicate characters

```
text = "programming"
result = "".join(dict.fromkeys(text))

print(result)  # progamin
```

Count words

```
sentence = "Python is easy to learn"

print(len(sentence.split()))  # 5
```

Longest word

```
sentence = "Python machine learning engineer"
words = sentence.split()

longest = max(words, key=len)
print(longest)  # learning
```

20. String and bytes

Strings represent text, while bytes represent binary data.

Encode: string to bytes

```
text = "Python"
data = text.encode("utf-8")

print(data)       # b'Python'
print(type(data))  # bytes
```

Decode: bytes to string

```
result = data.decode("utf-8")

print(result)       # Python
print(type(result))  # str
```

```
String → encode() → Bytes → decode() → String
```

21. Regular-expression revision

Regular expressions search and validate text patterns.

```
import re
```

Pattern	Meaning
\d	Digit
\w	Word character
\s	Whitespace
+	One or more
*	Zero or more
^	Beginning
$	End
[abc]	Any listed character
{n}	Exactly n times

```
text = "Order number is 45678"

print(re.findall(r"\d+", text))
# ['45678']
```

Mobile-number validation:

```
phone = "9876543210"

if re.fullmatch(r"[6-9]\d{9}", phone):
    print("Valid")
else:
    print("Invalid")
```

22. Time-complexity revision

Operation	Complexity
Access character	(O(1))

Operation	Complexity
len()	(O(1))
Slicing	(O(k))
Searching	(O(n))
Reversing	(O(n))
split()	(O(n))
replace()	(O(n))
Character frequency	(O(n))
Sorting characters	(O(n\log n))

23. Most important differences

find() versus index()

```
find()  → returns -1 when absent
index() → raises ValueError when absent
```

split() versus join()

```
split() → string to list
join()  → list of strings to string
```

== versus is

```
== → compares values
is → compares object identities
```

lower() versus casefold()

```
lower()    → normal lowercase conversion
casefold() → stronger Unicode-aware comparison
```

strip() versus replace()

```
strip()   → removes characters only from ends
replace() → replaces occurrences throughout the string
```

String versus list

```
String → immutable
List   → mutable
```

24. One-minute final revision

1. String means an immutable sequence of Unicode characters.
2. Indexing accesses one character.
3. Slicing extracts part of a string.
4. [::-1] reverses a string.
5. String methods return new strings.
6. len() returns character count.
7. split() creates a list.
8. join() creates one string.
9. strip() removes characters from the ends.
10. replace() replaces text.
11. find() returns an index or -1.
12. index() returns an index or raises an error.
13. startswith() and endswith() check boundaries.
14. isalpha(), isdigit() and isalnum() validate content.
15. casefold() supports reliable case-insensitive comparison.
16. == compares values; is compares identities.
17. encode() converts text to bytes.
18. decode() converts bytes to text.
19. Use Counter for frequency problems.
20. Two pointers, hashing and sliding windows are important string interview patterns.

You said:

most asked and conceptual for interview dsa string questions and solutions and easy word dry run

ChatGPT said:

Most-Asked DSA String Interview Questions with Python

Solutions

1. Important string patterns

Before solving a string problem, identify its pattern:

Problem type	Best pattern
Reverse or palindrome	Two pointers
Character counting	Hash map / Counter
Continuous substring	Sliding window
Opening and closing brackets	Stack
Same characters in different order	Frequency map / sorting
Matching a pattern	Hash maps
Longest palindrome	Expand around centre
Search one string inside another	Pattern matching
Generate combinations	Recursion / backtracking

Important difference:

- **Substring:** continuous characters, such as "yth" in "Python"
- **Subsequence:** characters follow the same order but may not be continuous, such as "Pto" in "Python"

2. Reverse a String

Question

Reverse a given string.

```
Input:  "python"
Output: "nohtyp"
```

Approach 1: Python slicing

```
def reverse_string(text):
    return text[::-1]


print(reverse_string("python"))
```

Complexity:

- Time: (O(n))
- Space: (O(n))

Approach 2: Two pointers

Because strings are immutable, convert the string into a list.

```
def reverse_string(text):
    characters = list(text)

    left = 0
    right = len(characters) - 1

    while left < right:
        characters[left], characters[right] = (
            characters[right],
            characters[left]
        )

        left += 1
        right -= 1

    return "".join(characters)


print(reverse_string("python"))
```

Dry run

Step	Left	Right	Operation	String
Initial	0	5	—	python
1	0	5	Swap p, n	nythop

Step	Left	Right	Operation	String
2	1	4	Swap y, o	nothy p → nothyp
3	2	3	Swap t, h	nohtyp

Complexity:

- Time: (O(n))
- Extra space: (O(n)) in Python because of the list

---

3. Valid Palindrome

Question

Check whether a string reads the same from left to right and right to left.

```
Input:  "Madam"
Output: True
```

A real interview question may ask you to ignore spaces, punctuation and case.

```
"A man, a plan, a canal: Panama" → True
```

Two-pointer solution

```
def is_palindrome(text):
    left = 0
    right = len(text) - 1

    while left < right:
        # Ignore non-alphanumeric characters
        while left < right and not text[left].isalnum():
            left += 1

        while left < right and not text[right].isalnum():
            right -= 1

        if text[left].casefold() != text[right].casefold():
            return False

        left += 1
        right -= 1

    return True


print(is_palindrome("Madam"))                    # True
print(is_palindrome("A man, a plan, a canal: Panama"))  # True
print(is_palindrome("Python"))                   # False
```

Dry run for "madam"

Left	Right	Comparison	Result
0 (m)	4 (m)	m == m	Continue
1 (a)	3 (a)	a == a	Continue
2 (d)	2 (d)	Pointers meet	True

Complexity:

- Time: (O(n))
- Space: (O(1))

4. Valid Anagram

Question

Two strings are anagrams if they contain the same characters with the same frequencies.

```
"listen" and "silent" → True
"hello" and "world"   → False
```

Frequency-map solution

```
def is_anagram(first, second):
    first = first.replace(" ", "").casefold()
    second = second.replace(" ", "").casefold()

    if len(first) != len(second):
        return False

    frequency = {}

    for character in first:
        frequency[character] = frequency.get(character, 0) + 1

    for character in second:
        if character not in frequency:
            return False

        frequency[character] -= 1

        if frequency[character] < 0:
            return False

    return all(count == 0 for count in frequency.values())


print(is_anagram("listen", "silent"))  # True
print(is_anagram("hello", "world"))    # False
```

Dry run

First string: "listen"

```
{
   'l': 1,
   'i': 1,
   's': 1,
   't': 1,
   'e': 1,
   'n': 1
}
```

Process "silent":

Character	Frequency after subtraction
s	s: 0
i	i: 0
l	l: 0
e	e: 0
n	n: 0
t	t: 0

All frequencies become zero, so the answer is True.

Complexity:

- Time: (O(n+m))
- Space: (O(k))
- k = number of unique characters

Short Python solution:

```
from collections import Counter


def is_anagram(first, second):
    return Counter(first.casefold()) == Counter(second.casefold())
```

---

5. Character Frequency

Question

Count the frequency of every character.

```
Input:  "banana"
Output: {'b': 1, 'a': 3, 'n': 2}
```

```
def character_frequency(text):
    frequency = {}

    for character in text:
        frequency[character] = frequency.get(character, 0) + 1

    return frequency


print(character_frequency("banana"))
```

Dry run

Character	Frequency dictionary
b	{'b': 1}
a	{'b': 1, 'a': 1}
n	{'b': 1, 'a': 1, 'n': 1}
a	{'b': 1, 'a': 2, 'n': 1}
n	{'b': 1, 'a': 2, 'n': 2}
a	{'b': 1, 'a': 3, 'n': 2}

Complexity:

- Time: (O(n))
- Space: (O(k))

6. First Non-Repeating Character

Question

Return the first character that appears only once.

```
Input:  "aabbcddee"
Output: "c"
```

Two-pass solution

```
def first_unique_character(text):
    frequency = {}

    # First pass: count characters
    for character in text:
        frequency[character] = frequency.get(character, 0) + 1

    # Second pass: preserve original order
    for index, character in enumerate(text):
        if frequency[character] == 1:
            return index, character

    return -1, None


print(first_unique_character("aabbcddee"))
# (4, 'c')
```

Why are two passes required?

The dictionary tells us which characters are unique. The second pass tells us which unique character appears first.

Dry run

Frequency map:

```
{
    'a': 2,
    'b': 2,
    'c': 1,
    'd': 2,
    'e': 2
}
```

Second pass:

Character	Frequency	Action
a	2	Skip
a	2	Skip
b	2	Skip
b	2	Skip
c	1	Return c

Complexity:

- Time: (O(n))
- Space: (O(k))

---

7. Remove Duplicate Characters

Question

Remove repeated characters while preserving the original order.

```
Input:  "programming"
Output: "progamin"
```

```
def remove_duplicates(text):
    seen = set()
    result =

    for character in text:
        if character not in seen:
            seen.add(character)
            result.append(character)

    return "".join(result)


print(remove_duplicates("programming"))
```

Dry run

Character	Already seen?	Result
p	No	p
r	No	pr
o	No	pro
g	No	prog
r	Yes	prog
a	No	proga
m	No	progam
m	Yes	progam
i	No	progami
n	No	progamin
g	Yes	progamin

Complexity:

- Time: (O(n))
- Space: (O(k))

8. Reverse Words in a Sentence

Question

Reverse the order of words.

```
Input:  "Python is very easy"
Output: "easy very is Python"
```

```
def reverse_words(sentence):
    words = sentence.split()
    words.reverse()

    return " ".join(words)


print(reverse_words("Python is very easy"))
```

Short solution:

```
def reverse_words(sentence):
    return " ".join(sentence.split()[::-1])
```

Dry run

```
Sentence
  ↓ split()
["Python", "is", "very", "easy"]
  ↓ reverse
["easy", "very", "is", "Python"]
  ↓ join()
"easy very is Python"
```

Complexity:

- Time: (O(n))
- Space: (O(n))

split() without an argument also removes unnecessary repeated spaces.

---

9. Longest Common Prefix

Question

Find the common beginning of every word.

```
Input:  ["flower", "flow", "flight"]
Output: "fl"
```

Solution

```
def longest_common_prefix(words):
    if not words:
        return ""

    prefix = words[0]

    for word in words[1:]:
        while not word.startswith(prefix):
            prefix = prefix[:-1]

            if prefix == "":
                return ""

    return prefix


print(longest_common_prefix(["flower", "flow", "flight"]))
```

Dry run

Current word	Prefix before	Operation	Prefix after
flower	—	Initial prefix	flower
flow	flower	Remove er	flow
flight	flow	Remove ow	fl

Result:

```
fl
```

Complexity:

- Time: approximately (O(n \times m))
- n = number of words
- m = length of the prefix
- Space: (O(1)), excluding returned string

10. Check String Rotation

Question

Check whether one string is a rotation of another.

```
"abcd" and "cdab" → True
"abcd" and "acbd" → False
```

Main concept

If second is a rotation of first, it must appear inside:

```
first + first
```

Example:

```
first        = abcd
first + first = abcdabcd
second       = cdab
```

"cdab" exists inside "abcdabcd".

Solution

```
def is_rotation(first, second):
    if len(first) != len(second):
        return False

    return second in (first + first)


print(is_rotation("abcd", "cdab"))  # True
print(is_rotation("abcd", "acbd"))  # False
```

Complexity:

- Time: generally (O(n)) using Python's optimized substring search
- Space: (O(n))

Important interview condition: both strings must have equal lengths.

11. Isomorphic Strings

Question

Two strings are isomorphic if every character in the first string maps consistently to exactly one character in the second string.

```
"egg" and "add" → True
"foo" and "bar" → False
```

For "egg" and "add":

```
e → a
g → d
```

Why use two dictionaries?

One dictionary prevents one source character from changing its mapping.
The second dictionary prevents two source characters from mapping to one target character.

```
def are_isomorphic(first, second):
    if len(first) != len(second):
        return False

    first_to_second = {}
    second_to_first = {}

    for char1, char2 in zip(first, second):
        if char1 in first_to_second:
            if first_to_second[char1] != char2:
                return False
        else:
            first_to_second[char1] = char2

        if char2 in second_to_first:
            if second_to_first[char2] != char1:
                return False
        else:
            second_to_first[char2] = char1

    return True


print(are_isomorphic("egg", "add"))  # True
print(are_isomorphic("foo", "bar"))  # False
```

Dry run for "egg" and "add"

Pair	Mapping
e → a	Valid new mapping
g → d	Valid new mapping
g → d	Matches existing mapping

Result: True

Complexity:

- Time: (O(n))
- Space: (O(k))

---

12. Valid Parentheses

Question

Check whether brackets are correctly opened and closed.

```
"(){}"  → True
"([{}])"  → True
"(]"    → False
"([)]"  → False
```

Stack solution

The most recently opened bracket must close first. This is called **Last In, First Out**, so a stack is used.

```
def valid_parentheses(text):
    stack = 

    matching = {
        ")": "(",
        "]": "[",
        "}": "{"
    }

    for bracket in text:
        if bracket in "([{":
            stack.append(bracket)

        elif bracket in matching:
            if not stack:
                return False

            top = stack.pop()

            if top != matching[bracket]:
                return False

    return len(stack) == 0


print(valid_parentheses("([{}])"))  # True
print(valid_parentheses("([)]"))    # False
```

Dry run for "([{}])"

Character	Action	Stack
(	Push	['(']
[	Push	['(', '[']
{	Push	['(', '[', '{']
}	Match and pop {	['(', '[']
]	Match and pop [	['(']
)	Match and pop (	``

Empty stack means all brackets are valid.

Complexity:

- Time: (O(n))
- Space: (O(n))

---

13. Longest Substring Without Repeating Characters

Question

Find the length of the longest continuous substring containing no repeated character.

```
Input:  "abcabcbb"
Output: 3
Substring: "abc"
```

Pattern: Sliding window

The window is represented by:

```
left ---------- right
```

When a repeated character appears, move left after the previous position of that character.

```
def longest_unique_substring(text):
    last_seen = {}

    left = 0
    best_start = 0
    best_length = 0

    for right, character in enumerate(text):
        if (
            character in last_seen
            and last_seen[character] >= left
        ):
            left = last_seen[character] + 1

        last_seen[character] = right
        current_length = right - left + 1

        if current_length > best_length:
            best_length = current_length
            best_start = left

    substring = text[best_start:best_start + best_length]
    return substring, best_length


print(longest_unique_substring("abcabcbb"))
# ('abc', 3)
```

Dry run

Right	Character	Left	Current window	Best
0	a	0	a	a
1	b	0	ab	ab
2	c	0	abc	abc
3	a	1	bca	abc
4	b	2	cab	abc
5	c	3	abc	abc
6	b	5	cb	abc

Right	Character	Left	Current window	Best
7	b	7	b	abc

Formula:

```
current_length = right - left + 1
```

Complexity:

- Time: (O(n))
- Space: (O(k))

---

14. Group Anagrams

Question

Group words that are anagrams.

```
Input:
["eat", "tea", "tan", "ate", "nat", "bat"]

Output:
[
   ["eat", "tea", "ate"],
   ["tan", "nat"],
   ["bat"]
]
```

Main concept

All anagrams produce the same sorted form.

```
eat → aet
tea → aet
ate → aet
```

Use the sorted form as a dictionary key.

```
from collections import defaultdict


def group_anagrams(words):
    groups = defaultdict(list)

    for word in words:
        key = "".join(sorted(word))
        groups[key].append(word)

    return list(groups.values())


words = ["eat", "tea", "tan", "ate", "nat", "bat"]
print(group_anagrams(words))
```

Dry run

Word	Sorted key	Dictionary group
eat	aet	aet: [eat]
tea	aet	aet: [eat, tea]
tan	ant	ant: [tan]
ate	aet	aet: [eat, tea, ate]
nat	ant	ant: [tan, nat]
bat	abt	abt: [bat]

Complexity:

- Time: (O(n \times k\log k))
- n = number of words
- k = maximum word length
- Space: (O(nk))

15. String Compression

Question

Compress consecutive repeated characters.

```
Input:  "aaabbccccd"
Output: "a3b2c4d1"
```

```
def compress_string(text):
    if not text:
        return ""

    result =
    count = 1

    for index in range(1, len(text)):
        if text[index] == text[index - 1]:
            count += 1
        else:
            result.append(text[index - 1] + str(count))
            count = 1

    # Add the final group
    result.append(text[-1] + str(count))

    return "".join(result)


print(compress_string("aaabbccccd"))
```

Why is the final append required?

Inside the loop, a group is added only when the character changes. The last group does not have another character after it, so it must be added separately.

Dry run

Current character	Previous	Count	Result
a	a	2	``
a	a	3	``
b	a	Reset to 1	['a3']
b	b	2	['a3']
c	b	Reset to 1	['a3', 'b2']
c,c,c	c	4	unchanged

Current character	Previous	Count	Result
d	c	Reset to 1	add c4
End	d	1	add d1

Complexity:

- Time: (O(n))
- Space: (O(n))

---

16. Longest Palindromic Substring

Question

Find the longest continuous substring that is a palindrome.

```
Input:  "babad"
Output: "bab" or "aba"
```

Main concept: Expand around centre

Every palindrome has a centre:

- Odd length: "racecar" has one centre character.

- Even length: "abba" has a centre between two characters.

```
def longest_palindrome(text):
    if not text:
        return ""

    best_start = 0
    best_end = 0

    def expand(left, right):
        while (
            left >= 0
            and right < len(text)
            and text[left] == text[right]
        ):
            left -= 1
            right += 1

        # Move back to the last valid palindrome
        return left + 1, right - 1

    for index in range(len(text)):
        # Odd-length palindrome
        odd_left, odd_right = expand(index, index)

        # Even-length palindrome
        even_left, even_right = expand(index, index + 1)

        if odd_right - odd_left > best_end - best_start:
            best_start, best_end = odd_left, odd_right

        if even_right - even_left > best_end - best_start:
            best_start, best_end = even_left, even_right

    return text[best_start:best_end + 1]


print(longest_palindrome("babad"))
print(longest_palindrome("cbbd"))  # bb
```

Dry run for "babad"

At centre index 1, character a:

```
Left = 1, Right = 1 → "a"
Left = 0, Right = 2 → "bab"
Left = -1 → Stop
```

At centre index 2, character b:

```
Left = 2, Right = 2 → "b"
Left = 1, Right = 3 → "aba"
Left = 0, Right = 4 → b != d, stop
```

Longest result can be "bab" or "aba".

Complexity:

- Time: (O(n^2))
- Space: (O(1))

---

17. Implement Substring Search

Question

Return the starting index of pattern inside text.

```
Text:    "machine learning"
Pattern: "learn"
Output:  8
```

Naive solution

```
def substring_search(text, pattern):
    if pattern == "":
        return 0

    if len(pattern) > len(text):
        return -1

    for start in range(len(text) - len(pattern) + 1):
        matched = True

        for offset in range(len(pattern)):
            if text[start + offset] != pattern[offset]:
                matched = False
                break

        if matched:
            return start

    return -1


print(substring_search("machine learning", "learn"))
```

Dry run

```
Text:    machine learning
Pattern: learn
```

Try each valid starting position:

```
machi → no
achin → no
chine → no
hine  → no
...
learn → match at index 8
```

Complexity:

- Time: (O(nm))

- Space: (O(1))

For advanced interviews, learn:

- KMP algorithm: (O(n+m))
- Rabin–Karp
- Z algorithm

---

18. String to Integer: atoi

Question

Convert a string into an integer without directly using int() for the conversion logic.

```
"42"        → 42
"   -42abc" → -42
"words 123" → 0
```

```
def string_to_integer(text):
    index = 0
    length = len(text)

    # Step 1: Ignore leading spaces
    while index < length and text[index] == " ":
        index += 1

    # Step 2: Read optional sign
    sign = 1

    if index < length and text[index] in "+-":
        if text[index] == "-":
            sign = -1
        index += 1

    # Step 3: Read digits
    number = 0

    while index < length and text[index].isdigit():
        digit = ord(text[index]) - ord("0")
        number = number * 10 + digit
        index += 1

    return sign * number


print(string_to_integer("42"))        # 42
print(string_to_integer("   -42abc")) # -42
print(string_to_integer("words 123")) # 0
```

Main formula

```
number = number * 10 + digit
```

Dry run for "123"

Character	Old number	Calculation	New number
1	0	0 × 10 + 1	1
2	1	1 × 10 + 2	12

Character	Old number	Calculation	New number
3	12	12 × 10 + 3	123

Complexity:

- Time: (O(n))
- Space: (O(1))

In LeetCode's version, also clamp the value to the 32-bit signed-integer range.

---

19. Minimum Window Substring

Question

Find the smallest substring of text containing all characters of target, including their required frequencies.

```
Text:   "ADOBECODEBANC"
Target: "ABC"
Output: "BANC"
```

Pattern: Variable sliding window

1. Expand the right pointer until all required characters are present.
2. Move the left pointer to make the window smaller.

3. Save the smallest valid window.

```
from collections import Counter


def minimum_window(text, target):
    if not text or not target:
        return ""

    required = Counter(target)
    window = {}

    required_types = len(required)
    formed_types = 0

    left = 0
    best_length = float("inf")
    best_start = 0

    for right, character in enumerate(text):
        window[character] = window.get(character, 0) + 1

        if (
            character in required
            and window[character] == required[character]
        ):
            formed_types += 1

        while formed_types == required_types:
            current_length = right - left + 1

            if current_length < best_length:
                best_length = current_length
                best_start = left

            left_character = text[left]
            window[left_character] -= 1

            if (
                left_character in required
                and window[left_character] < required[left_character]
            ):
                formed_types -= 1

            left += 1

    if best_length == float("inf"):
        return ""

    return text[best_start:best_start + best_length]
```

```
print(minimum_window("ADOBECODEBANC", "ABC"))
# BANC
```

Simplified dry run

Window	Contains A, B and C?	Action
ADOBEC	Yes	Save, then shrink
DOBEC	No A	Expand
CODEBA	Yes	Shrink
ODEBANC	Yes	Shrink
DEBANC	Yes	Shrink
EBANC	Yes	Shrink
BANC	Yes	Save smallest
ANC	No B	Stop shrinking

Complexity:

- Time: (O(n+m))
- Space: (O(k))

20. Decode String

Question

Decode an encoded expression.

```
Input:  "3[a2[c]]"
Output: "accaccacc"
```

Explanation:

```
2[c]     → cc
a2[c]    → acc
3[a2[c]] → accaccacc
```

Stack-style solution

```
def decode_string(encoded):
    count_stack =
    string_stack =

    current_string = ""
    current_number = 0

    for character in encoded:
        if character.isdigit():
            current_number = current_number * 10 + int(character)

        elif character == "[":
            count_stack.append(current_number)
            string_stack.append(current_string)

            current_number = 0
            current_string = ""

        elif character == "]":
            repeat_count = count_stack.pop()
            previous_string = string_stack.pop()

            current_string = (
                previous_string
                + current_string * repeat_count
            )

        else:
            current_string += character

    return current_string


print(decode_string("3[a2[c]]"))
```

Dry run

Character	Action	Current string
3	Number = 3	""

Character	Action	Current string
[	Save 3	""
a	Add character	"a"
2	Number = 2	"a"
[	Save 2 and "a"	""
c	Add character	"c"
]	Repeat "c" twice	"acc"
]	Repeat "acc" three times	"accaccacc"

Complexity:

- Time: (O(n + \text{output size}))
- Space: (O(n + \text{output size}))

Conceptual String Interview Questions

1. Why are Python strings immutable?

Once a string is created, its characters cannot be changed directly.

```
text = "Python"
# text[0] = "J"  # TypeError
```

Immutability makes strings:

- Safe to use as dictionary keys
- Safe to share
- Easier to hash
- More predictable

To modify one:

```
text = "J" + text[1:]
```

---

2. Why can a string be used as a dictionary key?

Dictionary keys must be hashable. Strings are immutable, so their hash value does not change after creation.

```
student = {
    "name": "Aamir",
    "role": "AI/ML Engineer"
}
```

Lists cannot be dictionary keys because lists are mutable.

3. What is the difference between == and is?

```
first == second
```

Checks whether values are equal.

```
first is second
```

Checks whether both variables refer to the exact same object.

Use == for string comparison.

4. What is the difference between substring and subsequence?

Concept	Continuous?	Example in "abcdef"
Substring	Yes	"bcd"
Subsequence	Not required	"ace"

Every substring is a subsequence, but every subsequence is not a substring.

5. What is the difference between palindrome and anagram?

- Palindrome: same forward and backward, such as "madam".
- Anagram: same character frequencies in a different order, such as "listen" and "silent".

6. Why use a hash map for string problems?

A hash map provides average (O(1)) lookup and update.

It helps with:

- Character frequency
- Duplicate detection
- Anagrams
- First unique character
- Sliding-window problems

7. When should we use two pointers?

Use two pointers when working from both ends or comparing pairs.

Examples:

- Reverse a string
- Palindrome checking
- Reverse vowels
- Compare strings after processing

8. When should we use a sliding window?

Use a sliding window when the question asks about a continuous substring.

Keywords:

- Longest substring
- Smallest substring
- At most `k` unique characters
- Without repeating characters
- Contains all required characters

9. Why is `"".join(result)` better than repeated concatenation?

Strings are immutable. Repeated concatenation may create many temporary strings.

Less suitable for large loops:

```
result = ""

for character in text:
    result += character
```

Preferred:

```
result =

for character in text:
    result.append(character)

answer = "".join(result)
```

10. Why does palindrome use left < right?

Only half of the string needs to be checked. Once the pointers meet or cross, every required pair has already matched.

Most Important Complexity Table

Problem	Best pattern	Time	Space
Reverse string	Two pointers	(O(n))	(O(n)) in Python
Valid palindrome	Two pointers	(O(n))	(O(1))
Valid anagram	Hash map	(O(n))	(O(k))
Character frequency	Hash map	(O(n))	(O(k))
First unique character	Hash map	(O(n))	(O(k))
Remove duplicates	Set	(O(n))	(O(k))
Valid parentheses	Stack	(O(n))	(O(n))
Longest unique substring	Sliding window	(O(n))	(O(k))
Group anagrams	Sorting + hash map	(O(nk\log k))	(O(nk))
Longest palindrome	Centre expansion	(O(n^2))	(O(1))

Problem	Best pattern	Time	Space
Naive pattern search	Nested loops	(O(nm))	(O(1))
Minimum window	Sliding window	(O(n+m))	(O(k))

Best Revision Order

Study these first:

1. Reverse string
2. Valid palindrome
3. Valid anagram
4. Character frequency
5. First non-repeating character
6. Reverse words
7. Longest common prefix
8. String rotation
9. Valid parentheses
10. Longest substring without repeating characters
11. Group anagrams
12. String compression
13. Longest palindromic substring
14. String-to-integer conversion
15. Minimum window substring

Final interview rule:

> If the question asks about character counts, use a hash map. If it asks about a continuous substring, think of a sliding window. If it compares both ends, use two pointers. If it contains matching brackets, use a stack.